

Spark SQL and DataFrame Processing

1. Spark SQL Overview

Spark SQL is Spark's module for structured data processing. It supports SQL queries and DataFrame operations and provides a common execution engine for relational workloads.

Tables can be temporary views, managed tables, external tables, or views backed by other data sources such as Parquet, JSON, CSV, JDBC, and Delta-style storage systems.

2. DataFrames and Schemas

A DataFrame is a distributed table with named columns. Schemas can be inferred or explicitly specified. Explicit schemas are often preferable for production ingestion because they reduce ambiguity and make data contracts clearer.

3. Reading and Writing Data

```
df = spark.read.parquet("/data/events")
df.createOrReplaceTempView("events")

result = spark.sql("""
    SELECT event_type, COUNT(*) AS event_count
    FROM events
    GROUP BY event_type
""")

result.write.mode("overwrite").parquet("/data/output/event_counts")
```

4. Filtering and Projection

Projection selects columns while filtering restricts rows. Predicate pushdown can allow filters to be applied closer to the data source, reducing the amount of data read.

Selecting only required columns is also useful because column pruning can reduce I/O.

5. Joins

Spark supports inner, left, right, full, semi, and anti joins. Join performance depends heavily on data size, partitioning, key distribution, and whether one side can be broadcast.

A broadcast join can be effective when one dataset is sufficiently small. Spark can distribute the smaller relation to executors so the larger relation does not need a full shuffle join.

6. Aggregations

Aggregations such as `groupBy` and `count` commonly require data to be redistributed by grouping keys. High-cardinality grouping keys can create many partitions, while skewed keys can cause a small number of tasks to process disproportionately large amounts of data.

7. Window Functions

Window functions calculate values across related rows without collapsing them into a single row per group. Common functions include `row_number`, `rank`, `dense_rank`, `lag`, `lead`, and running aggregates.

Window specifications typically define partitioning and ordering. Large windows can be expensive because they may require sorting and data movement.

8. Common SQL Optimization Techniques

Use filters early, select only required columns, avoid unnecessary distinct operations, choose appropriate join strategies, inspect execution plans, and avoid repeated scans of expensive source data.

Use `EXPLAIN` to understand logical and physical plans.

9. Example Window Query

```
SELECT
  customer_id,
  transaction_date,
  amount,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY transaction_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM transactions;
```

10. Useful Concepts

Catalyst, logical plan, physical plan, predicate pushdown, column pruning, broadcast join, shuffle join, sort-merge join, aggregation, window function, partition pruning, `EXPLAIN`.